

إدارة شبكة المدينة الذكية

(Smart Grid Manager)

محاكاة لتوزيع الطاقة باستخدام أنماط التصميم (Design Patterns)

تقديم الطلبة:

حيدر أحمد الجنفاوي

سالم محمد الصيد

أحمد منصور راشد

نظرة عامة على المشروع

مفهوم النظام والمشكلات التقنية التي يعالجها

الفكرة العامة للمشروع



محاكاة ذكية: نظام تفاعلي لإدارة وتوزيع الطاقة الكهربائية في بيئة "مدينة ذكية".



تقسيم المناطق: توزيع كمية طاقة متغيرة ديناميكياً على مناطق مختلفة كالمستشفيات، الأحياء السكنية، والمصانع.



قرارات متكيفة: الاعتماد على نظام يتخذ قرارات التوزيع وفق "استراتيجيات" مرنة تتغير بناءً على حالة توفر الطاقة.



تحكم لحظي: توفير واجهة رسومية تفاعلية للمستخدم لتغيير استراتيجيات التوزيع أثناء وقت التشغيل (Runtime).



أنماط التصميم المستخدمة (Design Patterns)



Observer Pattern (نمط المراقب)

استخدم لربط التحديثات الخلفية للنظام بالواجهة الرسومية بشكل مباشر ولحظي دون كتابة كود متشابك ومعقد.



Strategy Pattern (نمط الاستراتيجية)

استخدم لفصل خوارزميات توزيع الطاقة في كلاسات مستقلة، لتوفير طرق توزيع مرنة ومتعددة تتغير حسب ظروف الشبكة وقت التشغيل.



Singleton Pattern (نمط التفرد)

استخدم لضمان وجود "مدير تحكم مركزي واحد فقط" لشبكة الكهرباء، مما يمنع تعدد النسخ ويتجنب التضارب في أرقام وتوزيع الطاقة.

1. Singleton Pattern (نمط التفرد)



التركيز التقني: كلاس GridManager

🎯 **السبب:** شبكة الكهرباء في أي مدينة حقيقية تتطلب تحكماً مركزياً صارماً. تعدد المديرين سيؤدي حتماً إلى فوضى في أرقام التوزيع.

⚙️ آلية العمل:

- تم جعل الـ Constructor خاصاً (Private) لمنع إنشاء كائنات جديدة عشوائياً.
- تم توفير الدالة getInstance() والتي تضمن دائماً إعادة نفس النسخة من المدير لكل أجزاء النظام.

2. Strategy Pattern (نمط الاستراتيجية)

تم تطبيقه عبر واجهة DistributionStrategy لتفادي جمل if-else المعقدة، وتقديم 3 خوارزميات نقية:

الاسم الاستراتيجية	المناطق الحرجة (المستشفيات)	باقي المناطق (سكنية/مصانع)	الهدف من الاستراتيجية
NormalStrategy (العادي)	تستلم 100% من احتياجها	تستلم 100% من احتياجها	الاستقرار وتوزيع وفرة الطاقة
EcoStrategy (الاقتصادي)	تستلم 100% من احتياجها	تستلم 70% فقط (خصم 30%)	توفير الطاقة مع الحفاظ على التشغيل الأساسي
EmergencyStrategy (الطوارئ)	توجيه كل الطاقة 100% لها	يتم قطع الكهرباء تماماً 0%	إنقاذ الأرواح في أوقات العجز الشديد

3. Observer Pattern (نمط المراقب)



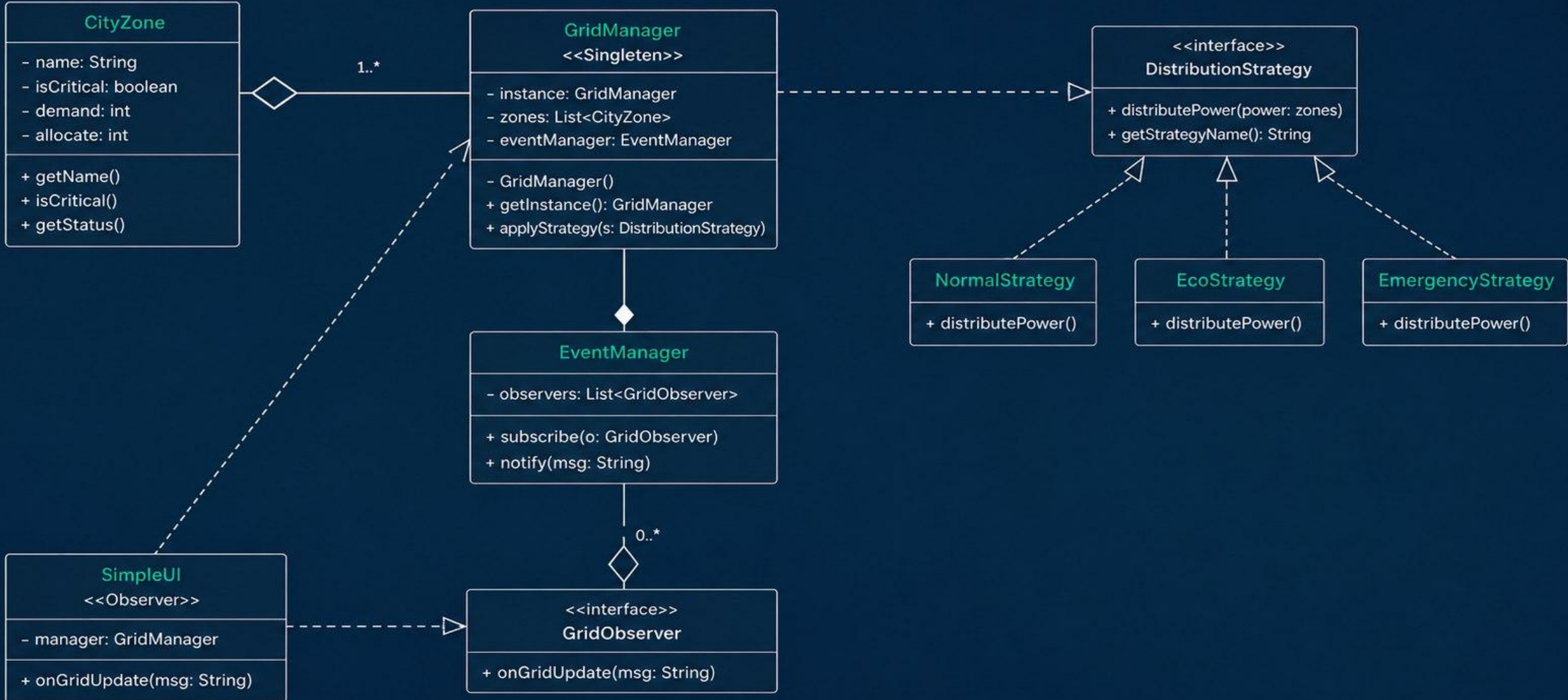
الناشر: EventManager | المراقب: SimpleUI

السبب: نحتاج لتحديث الشاشة وسجل الأحداث (Log) فوراً عند نقص الطاقة أو تغيير الاستراتيجية، دون أن نجعل كود المدير (Backend) معتمداً كلياً على واجهة المستخدم (Frontend).

آلية العمل: الواجهة الرسومية تشترك (Subscribe) في نظام الأحداث. عندما يوزع GridManager الطاقة، يرسل إشعاراً لـ EventManager، والذي بدوره ينبه الواجهة فوراً لتحديث السجل على الشاشة.

مخطط الكلاسات (UML)

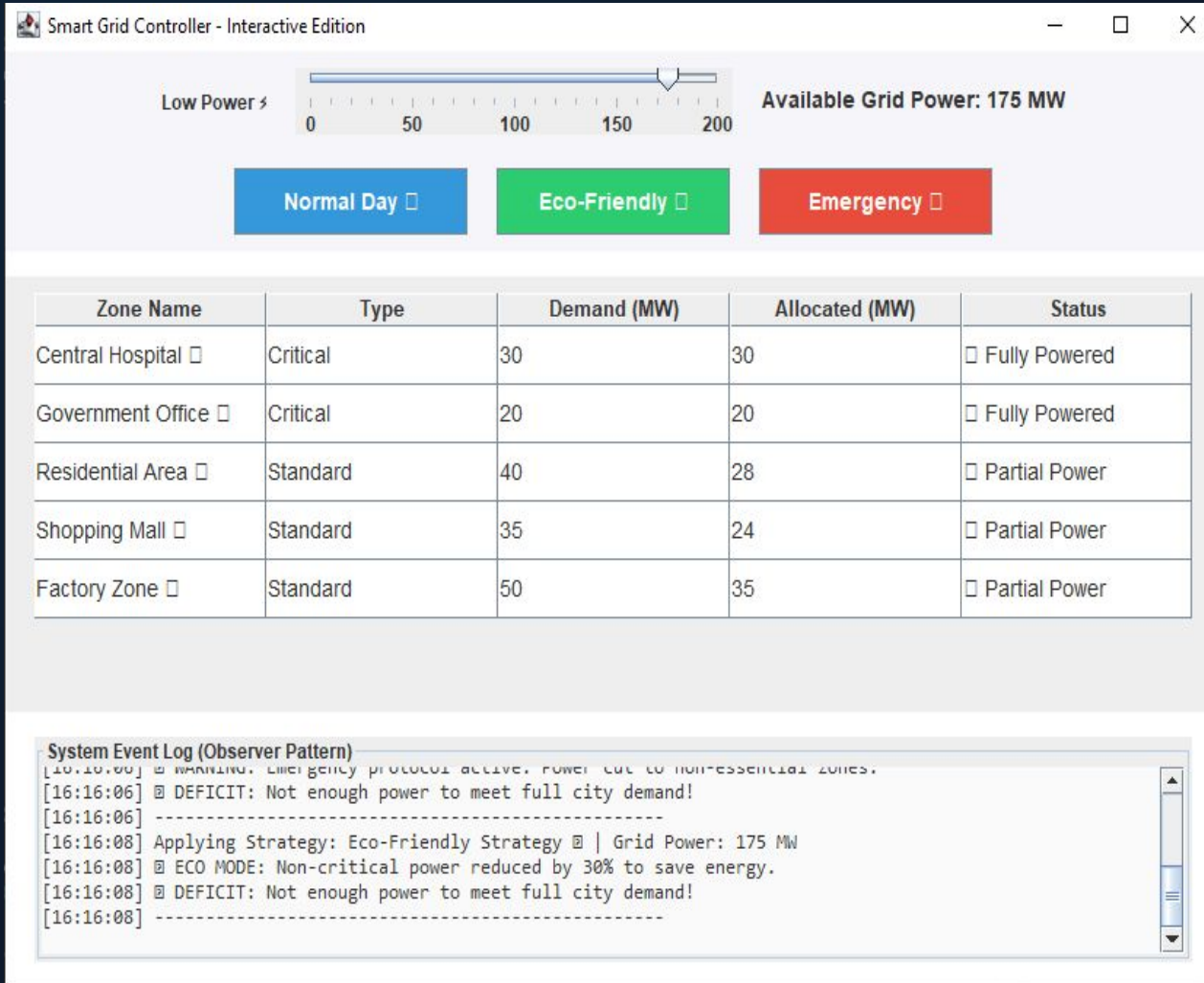
يوضح الكلاسات والعلاقات بينها في النظام



مخطط الكلاسات والعلاقات (UML Relationships)

- ◆ **Composition (تكوين)** ◆: علاقة قوية جداً بين GridManager و EventManager. مدير الشبكة هو من يقوم بإنشاء نظام التنبيهات، وهو جزء حيوي لا يتجزأ منه (إذا دُمِر المدير يُدمر الناشر).
- ◆ **Aggregation (تجميع)** ◆: علاقة تجميع بين GridManager والمناطق CityZone. المدير يمتلك قائمة من المناطق، ولكن المناطق تعتبر كيانات مستقلة يمكنها البقاء بدونه.
- ↑ **Realization (تحقيق)** □: استراتيجيات التوزيع (Normal, Eco, Emergency) تقوم بتطبيق وتحقيق الواجهة DistributionStrategy. وكذلك الواجهة الرسومية تطبق الانترفيس GridObserver.
- ← **Association (ارتباط)** □: كلاس GridManager يرتبط بواجهة التوزيع لتنفيذ الأوامر حسب الخيار الذي يعينه المستخدم في وقت التشغيل.

الواجهة الرسومية التفاعلية (User Interface)



لوحة تحكم مبنية بـ Java Swing

شريط تحكم (Slider): يتيح لك تغيير "كمية الطاقة

المتوفرة حالياً" لمحاكاة أوقات الذروة أو العجز.

أزرار الاستراتيجيات: التبديل الفوري بين الوضع

(العادي، الاقتصادي، الطوارئ).

جدول البيانات الحي: يعرض فوراً حالة كل منطقة

بناءً على حصتها من الطاقة.

سجل الأحداث (Event Log): شاشة مدمجة تطبع

التقارير وحالة النظام لحظياً بالاعتماد على نمط

المراقب (Observer).

هل لديكم أي أسئلة؟

شكراً لاستماعكم واهتمامكم

Smart Grid Manager: مشروع

(Design Patterns) مادة أنماط البرمجيات